



PROJECT REPORT

TITLE:

**SMART MULTI-VENDOR MARKETPLACE WITH
VENDOR PERFORMANCE & SALES ANALYTICS**

COURSE: ADVANCED DBMS LAB

GROUP MEMBERS:

SYED ASJAD ALI- 240201086

M. HASNAIN ALI-240201060

ABDUL NABI-240201082

SECTION: CS-05B

DEPARTMENT OF COMPUTER SCIENCE

Table of Contents

Introduction.....	2
Objectives.....	2
Requirements Analysis	3
Functional Requirements	3
Non-Functional Requirements	3
MongoDB Implementation.....	3
Database Design.....	3
CRUD Operations	4
API responses.....	5
Aggregation Pipelines	6
Views and Materialized Views.....	8
Embedded Documents and References	10
Performance Optimization.....	11
Indexing	11
Query Optimization	13
Security and Scalability.....	13
Database Security.....	13
Concurrency Handling	14
Scalability Concepts.....	14
Application Interface	15
GitHub Repository.....	16
Challenges and Lessons Learned.....	16
Conclusion and Future Enhancements	17

1. Introduction

The Smart Multi-Vendor Marketplace Management System is a web-based application developed using MongoDB, Express.js, React.js, and Node.js. The system helps manage products, vendors, and customer orders efficiently in an e-commerce environment.

MongoDB was selected because it provides flexible document storage, fast performance, and easy scalability. The project includes important MongoDB features such as CRUD operations, aggregation pipelines, indexing, query optimization, views, and MongoDB Atlas integration.

The system also provides a user-friendly dashboard with analytics, searching, filtering, and vendor performance monitoring. MongoDB Compass, Atlas, and MongoDB Shell were used for database management and analytics operations.

2. Objectives

The main objectives of this project are:

- To develop a complete multi-vendor marketplace management system.
- To implement MongoDB database operations in a practical environment.
- To perform CRUD operations using Express.js and MongoDB.
- To design an interactive frontend dashboard using React.js.
- To implement aggregation pipelines for analytical reporting.
- To optimize database performance using indexes.
- To integrate MongoDB Atlas cloud database.
- To implement references and embedded document concepts.
- Managing vendor products and customer orders efficiently.
- To provide product search and filtering functionality.
- To generate visual analytical reports using charts.
- To understand scalability and security concepts in MongoDB.
- To improve backend and frontend integration skills.

3. Requirements Analysis

Functional Requirements

The system supports product, vendor, and order management operations. Users can add, edit, delete, search, and filter products. Vendor analytics and revenue calculations are also supported. The system allows order placement, stock updates, and prevents orders when stock is insufficient. Analytics and graphical reports are generated using aggregation pipelines and charts.

MongoDB collections, references, indexing, views, and aggregation pipelines were implemented for efficient database management.

Non-Functional Requirements

The system was designed for good performance, security, scalability, and usability. Indexing and optimized queries improved response speed, while MongoDB Atlas authentication provided secure database access.

The frontend dashboard is responsive and user-friendly. Stock validation ensures data reliability, and the modular project structure supports future maintenance and scalability.

4. MongoDB Implementation

Database Design

The project uses MongoDB as the primary database system. MongoDB stores data in document format using collections. MongoDB Atlas was used as a cloud database service. MongoDB Compass and MongoDB Shell were used for testing and database operations.

The main collections created were products, orders, and vendors.

```
_id: ObjectId('6a01e970482ec513d508e33c')
customerName : "Asjad Ali"
productName : "iPhone 11"
quantity : 1
totalPrice : 40500
vendorName : "Apple Store"
createdAt : 2026-05-11T14:36:32.090+00:00
updatedAt : 2026-05-11T14:36:32.090+00:00
__v : 0
```

Product Collection

The products collection stores marketplace product information.

Product Fields: productName, price, stock, category, vendorName, createdAt, updatedAt.

Order Collection

The orders collection stores customer order details.

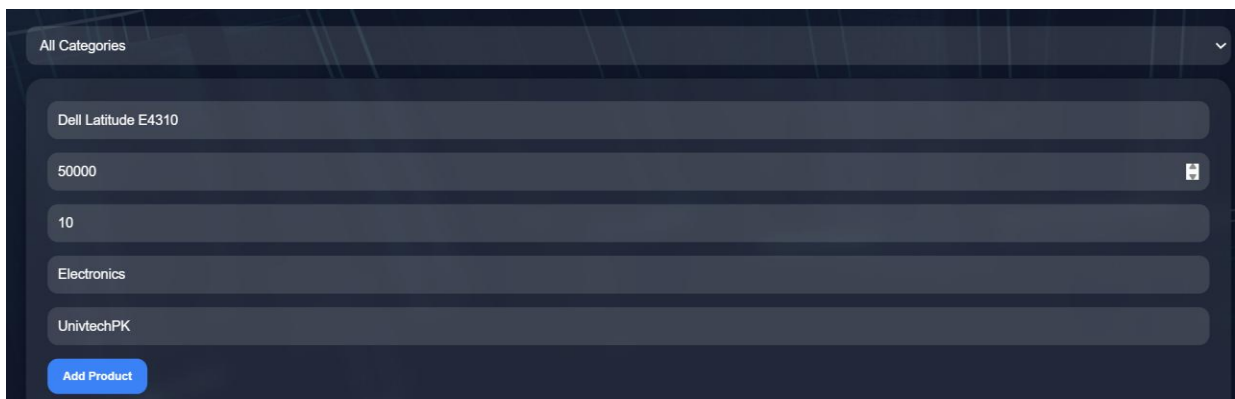
Order Fields: _id (means Product Id), quantity, customerName, orderDate, vendorName, totalPrice, quantity.

The order system also updates stock quantity automatically after successful order placement.

CRUD Operations

CRUD operations were implemented using Express.js routes and MongoDB queries. Products and orders can be added, viewed, updated, and deleted through the frontend dashboard and backend APIs.

- **Add product operation.**



A dark-themed form for adding a product. It features a dropdown menu at the top labeled 'All Categories'. Below it are five input fields containing the text: 'Dell Latitude E4310', '50000', '10', 'Electronics', and 'UnivtechPK'. At the bottom left of the form is a blue button labeled 'Add Product'.

After adding



A dark-themed card displaying the details of the added product. The title 'Dell Latitude E4310' is in blue. Below it, the details are listed: 'Price: Rs. 50000', 'Stock: 10', 'Category: Electronics', and 'Vendor: UnivtechPK'. At the bottom are two blue buttons: 'Edit' and 'Delete'.

- **Edit product operation.**

Dell Latitude E4310

50000

8

Electronics

UnivtechPK

Update Product

After updating

Dell Latitude E4310

Price: Rs. 50000

Stock: 8

Category: Electronics

Vendor: UnivtechPK

Edit Delete

API responses

```
localhost:5000/api/products

{
  "success": true,
  "products": [
    {
      "_id": "6a007aea1246246741ad4eb1",
      "productName": "iPhone 13 Pro",
      "price": 130000,
      "stock": 4,
      "category": "Mobiles",
      "vendorName": "Apple Store",
      "createdAt": "2026-05-10T12:32:42.285Z",
      "updatedAt": "2026-05-16T09:41:12.210Z",
      "__v": 0
    },
    {
      "_id": "6a0082cb24e8e3e631888c5a",
      "productName": "HP Elitebook 845 G10",
      "price": 270000,
      "stock": 7,
      "category": "Laptops",
      "vendorName": "Univtech PK",
      "createdAt": "2026-05-10T13:06:19.346Z",
      "updatedAt": "2026-05-11T14:18:54.663Z",
      "__v": 0
    },
    {
      "_id": "6a01e2316043c1713c4f44f3",
      "productName": "Samsung S24",
      "price": 80000,
      "stock": 10,
      "category": "Mobiles",
      "vendorName": "Galaxy Store",
      "createdAt": "2026-05-11T14:05:37.608Z",
      "updatedAt": "2026-05-11T14:05:37.608Z",
      "__v": 0
    },
    {
      "_id": "6a01e2706043c1713c4f44f4",
      "productName": "Eyeliner Ultra thin",
      "price": 1150,
      "stock": 24,
      "category": "Fashion",
      "vendorName": "Rivaj",
      "createdAt": "2026-05-11T14:06:40.247Z",
      "updatedAt": "2026-05-11T14:06:40.247Z",
      "__v": 0
    },
    {
      "_id": "6a01e2c96043c1713c4f44f5",
      "productName": "Women Handbag Aesthetic",
      "price": 3800,
      "stock": 5,

```

Aggregation Pipelines

Aggregation pipelines were implemented for analytics and reporting purposes.

Aggregation operators including \$match, \$group, \$sum, \$avg, \$sort, \$project, and \$lookup were used.

1. Vendor Revenue Analytics

Aggregation pipelines were used to calculate vendor revenue. The system grouped products vendor-wise and calculated total inventory value.

```
> use smartMarketplaceDB
< switched to db smartMarketplaceDB
> db.orders.aggregate([

  {
    $group: {

      _id: null,

      totalRevenue: {
        $sum: "$totalPrice"
      }
    }
  }

])
< {
  _id: null,
  totalRevenue: 311650
}
```

2. Product Category Analytics

Products were grouped according to categories to generate category-based analytics.

```

> db.products.aggregate([
  {
    $group: {
      _id: "$category",
      totalProducts: {
        $sum: 1
      }
    }
  }
])
< {
  _id: 'Fashion',
  totalProducts: 1
}
{
  _id: 'Accessories',
  totalProducts: 5
}

```

3. Total orders per vendor:

```

> db.orders.aggregate([
  {
    $group: {
      _id: "$vendorName",
      totalOrders: {
        $sum: 1
      }
    }
  }
])
< {
  _id: 'Apple Store',
  totalOrders: 1
}
{
  _id: 'Rivaj',
  totalOrders: 1
}
{
  _id: 'Univtech PK',
  totalOrders: 1
}

```

```

< {
  _id: 'Univtech PK',
  totalRevenue: 270000
}

```


Views and Materialized Views

MongoDB views were created for analytics and reporting. Views simplified complex aggregation queries and improved report generation.

View 1- vendorSummaryview:

```
> db.createView(
  "vendorSummaryView",
  "products",
  [
    {
      $group: {
        _id: "$vendorName",
        totalProducts: {
          $sum: 1
        },
        totalStock: {
          $sum: "$stock"
        },
        totalInventoryValue: {
          $sum: {
            $multiply: [
              $price,
              $stock
            ]
          }
        }
      }
    }
  ]
)
```

Output:

```
< { ok: 1 }
> show collections
< products
  vendorSummaryView [view]
```

View 2- Category Sales View:

```
> db.createView(
  "categoryAnalyticsView",
  "products",
  [
    {
      $group: {
        _id: "$category",
        totalProducts: {
          $sum: 1
        },
        totalStock: {
          $sum: "$stock"
        },
        averagePrice: {
          $avg: "$price"
        }
      }
    }
  ]
)

< { ok: 1 }
> db.categoryAnalyticsView.find()
< {
  _id: 'Fashion',
  totalProducts: 1,
  totalStock: 24,
  averagePrice: 1150
}
{
  _id: 'Accessories',
  totalProducts: 5,
  totalStock: 48,
  averagePrice: 2050
}
{
  _id: 'Mobiles',
  totalProducts: 4,
  totalStock: 28,
  averagePrice: 73875
}
{
  _id: 'Laptops',
  totalProducts: 1,
  totalStock: 7,
  averagePrice: 270000
}
]
```

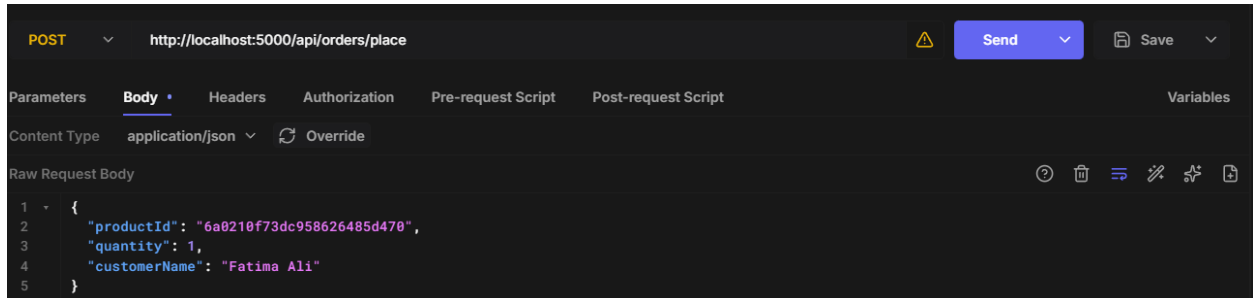
View 3- Low Stock View:

```
> db.createView(
  "lowStockProductsView",
  "products",
  [
    {
      $match: {
        stock: {
          $lt: 5
        }
      }
    }
  ]
)

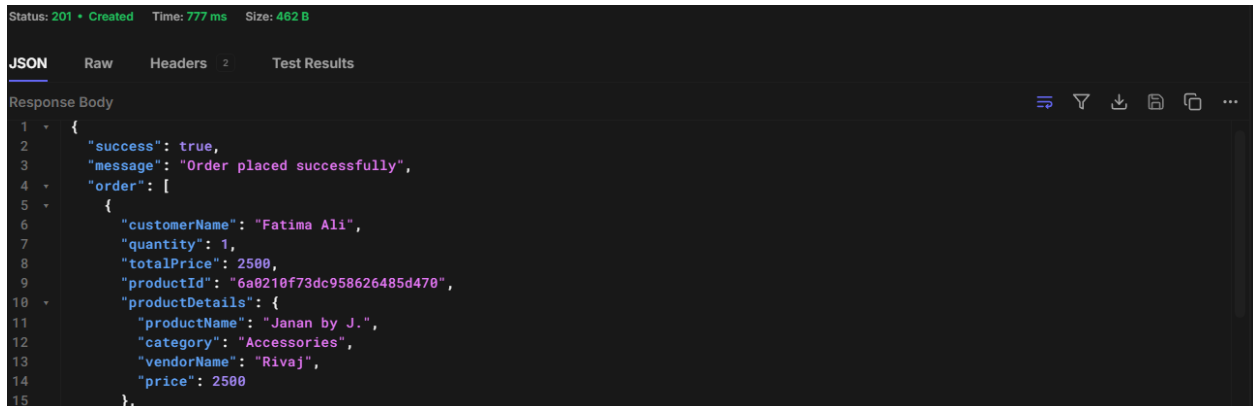
< { ok: 1 }
> db.lowStockProductsView.find()
< {
  _id: ObjectId('6a007aea1246246741ad4eb1'),
  productName: 'iPhone 13 Pro',
  price: 130000,
  stock: 4,
  category: 'Mobiles',
  vendorName: 'Apple Store',
  createdAt: 2026-05-10T12:32:42.285Z,
  updatedAt: 2026-05-16T09:41:12.210Z,
  __v: 0
}
{
  _id: ObjectId('6a01e2fd6043c1713c4f44f6'),
  productName: 'iPhone 11',
  price: 40500,
  stock: 2,
  category: 'Mobiles',
  vendorName: 'Apple Store',
  createdAt: 2026-05-11T14:09:01.931Z,
  updatedAt: 2026-05-11T14:09:01.931Z,
  __v: 0
}
]
```

Embedded Documents and References

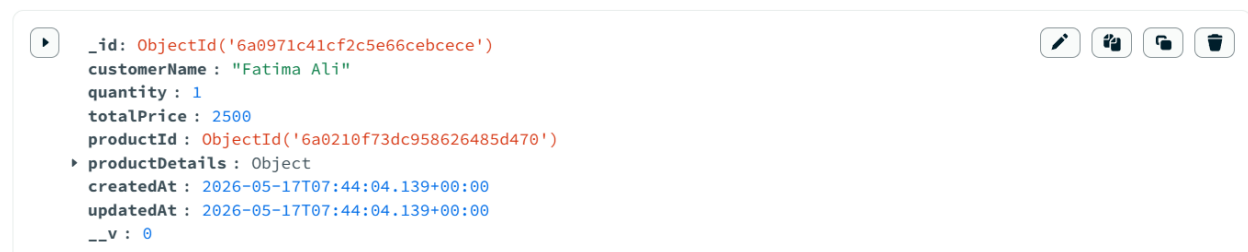
References were implemented between products and orders using productId. Orders reference products to maintain data relationships. Embedded document concepts were also studied during implementation.



Response:



Inserted in MongoDB too:



MongoDB Atlas cloud database was successfully integrated with the backend server using a secure connection string stored in the .env file.

The screenshot shows the MongoDB Data Explorer interface. On the left, the 'Data Explorer' sidebar is visible with a search bar and a list of clusters. The 'marketplaceCluster' cluster is selected, showing its sub-clusters: 'admin', 'local', 'smartMarketplaceDB', and 'test'. The main panel displays the 'marketplaceCluster' database structure with a table of databases and their properties.

Database name	Storage size	Data size	Collections	Indexes
admin	0 B	0 B	0	0
local	-	-	0	-
smartMarketplaceDB	126.98 kB	3.72 kB	7	12
test	73.73 kB	803.00 B	0	2

5. Performance Optimization

Performance optimization improved data retrieval and query execution efficiency. In this project, indexing and query optimization techniques were used to improve system performance.

Indexing

Indexes were created on important fields such as productName, category, vendorName.

Index 1- Product Name:

```
> db.products.createIndex({
  productName: 1
})
< productName_1
```

Index 2- Category:

```
> db.products.createIndex({
  category: 1
})
< category_1
```

Index 3- Vendor Name:

```
> db.products.createIndex({
  vendorName: 1
})
< vendorName_1
```

Index 4- Orders Vendor:

```
> db.orders.createIndex({  
  
  vendorName: 1  
  
})  
< vendorName_1
```

Index 5- Customer Name:

```
> db.orders.createIndex({  
  
  customerName: 1  
  
})  
< customerName_1
```

Index Product Verification:

```
> db.products.getIndexes()  
< [  
  { v: 2, key: { _id: 1 }, name: '_id_' },  
  { v: 2, key: { productName: 1 }, name: 'productName_1' },  
  { v: 2, key: { category: 1 }, name: 'category_1' },  
  { v: 2, key: { vendorName: 1 }, name: 'vendorName_1' }  
]
```

Index Order Verification:

```
> db.orders.getIndexes()  
< [  
  { v: 2, key: { _id: 1 }, name: '_id_' },  
  { v: 2, key: { vendorName: 1 }, name: 'vendorName_1' },  
  { v: 2, key: { customerName: 1 }, name: 'customerName_1' }  
]
```

Query Optimization

MongoDB explain plan was used for optimization.

Explain Query:

```
> db.products.find({
  category: "Electronics"
}).explain("executionStats")
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'smartMarketplaceDB.products',
    parsedQuery: {
      category: {
        '$eq': 'Electronics'
      }
    },
    indexFilterSet: false,
    queryHash: '421A7F3B',
    planCacheShapeHash: '421A7F3B',
    planCacheKey: 'E8986359',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      executionTimeMillisEstimate: 0,
      works: 1,
      advanced: 0,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      docsExamined: 0,
      alreadyHasObj: 0,
      inputStage: {
        stage: 'IXSCAN',
        nReturned: 0,
        executionTimeMillisEstimate: 0,
        works: 1,
        advanced: 0,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        keyPattern: {
          category: 1
        },
        indexName: 'category_1',
        isMultiKey: false,
```

6. Security and Scalability

Database Security

MongoDB Atlas authentication was used for secure database access. The database connection string was stored inside the .env file instead of directly writing credentials inside source code. This method improves application security and protects sensitive credentials. Security features included MongoDB Atlas authentication, environment variables (.env), secure database connections, and access control concepts.

Read-only and admin role concepts were also studied during development.

ORGANIZATION

Asjad's Org - 2026-0...

PROJECT

SemesterProject

Database Users

+ ADD NEW DATABASE USER

User	Description	Authentication Method	MongoDB Roles	Resources	Actions
manageruser		SCRAM	readWriteAnyDatabase@admin	All Resources	EDIT DELETE
readonlyuser		SCRAM	readAnyDatabase@admin	All Resources	EDIT DELETE
sasjadali572_db_user		SCRAM	atlasAdmin@admin	All Resources	EDIT DELETE

```

backend > .env
1 MONGO_URI=mongodb://sasjadali572_db_user: @ac-b37brt-shard-00-00.iyms1it.mongodb.net:27017
2 PORT=5000

```

Concurrency Handling

Concurrency handling was implemented through stock validation. Orders are rejected if stock is insufficient, otherwise stock quantity updates automatically to maintain consistency. This prevents inconsistent inventory data.

HOPPSCOTCH

Search and commands Ctrl K

POST

Untitled

Select environment

Send

Save

Parameters

Body

Headers

Authorization

Pre-request Script

Post-request Script

Variables

Content Type

application/json

Override

Raw Request Body

```

1 {
2   "productId": "6a907aea1246246741ad4eb1",
3   "quantity": 9999,
4   "customerName": "Ali"
5 }
6

```

Status: 500 Internal Server Error

Time: 349 ms

Size: 120 B

JSON

Raw

Headers

Test Results

Response Body

```

1 {
2   "success": false,
3   "message": "Insufficient stock"
4 }

```

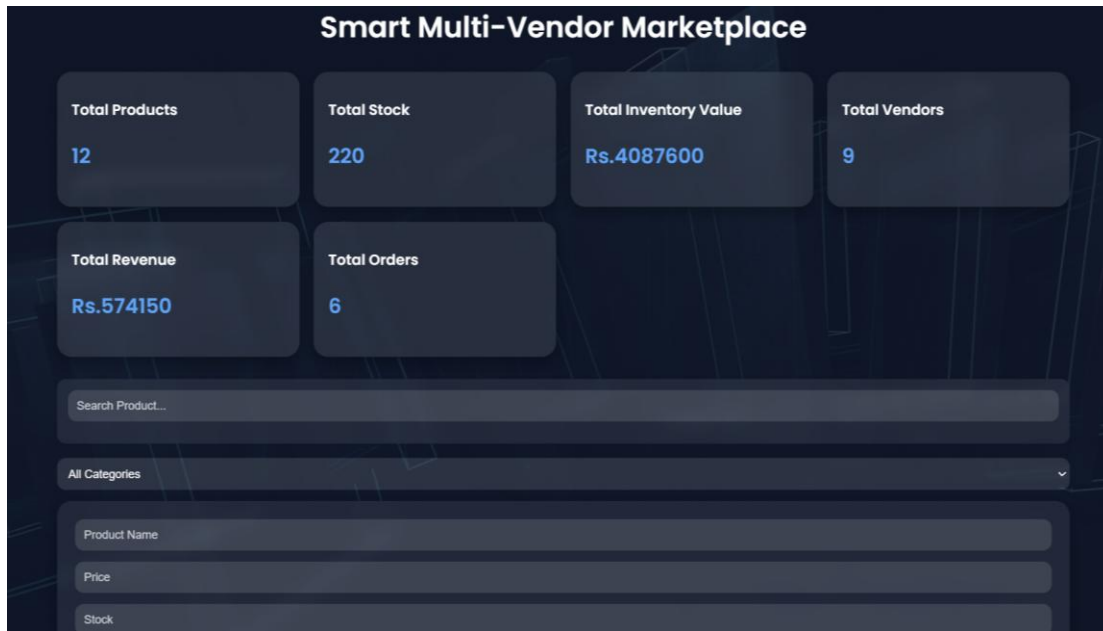
Scalability Concepts

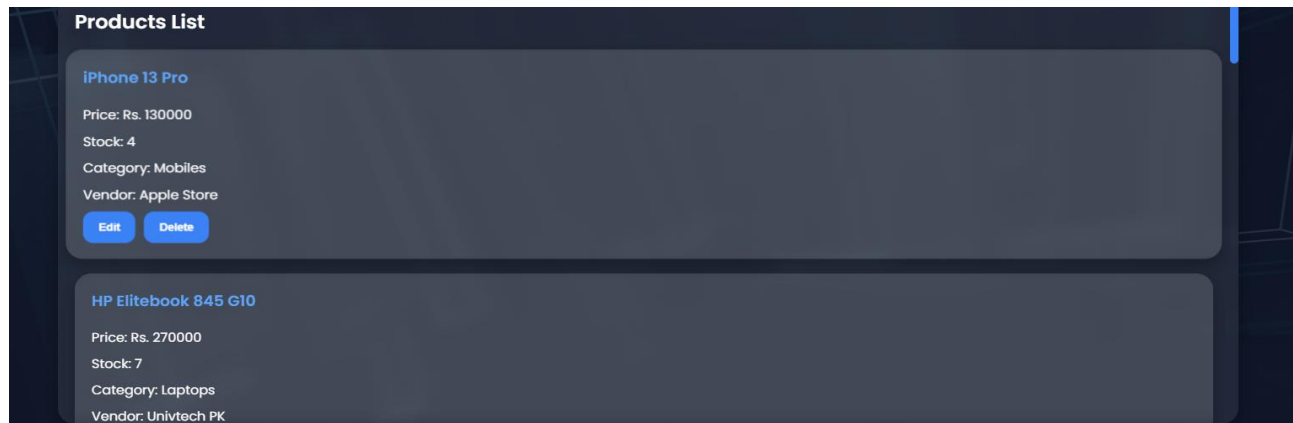
MongoDB Atlas cloud infrastructure supports scalability and large-scale deployment. Sharding concepts were studied theoretically but were not implemented because the project dataset was small. The project structure supports future scalability.

7. Application Interface

The Smart Multi-Vendor Marketplace provides a user-friendly interface developed using React.js. The dashboard allows users to manage products, orders, analytics, searching, and filtering through an interactive graphical interface. Analytical charts were implemented using the Recharts library for vendor revenue and inventory visualization.

The application workflow includes frontend requests, backend API processing using Express.js, MongoDB data storage, and dynamic result display on the dashboard.





8. GitHub Repository

The complete Smart Multi-Vendor Marketplace project was uploaded to GitHub repositories for proper version control and project management. GitHub was used for Code backup, Project sharing, Collaboration, Version tracking, and Documentation management. Each repository contains Frontend source code, Backend source code, MongoDB configuration, README documentation, Project structure.

GitHub repositories links:

<https://github.com/sasjadali572-prog/Smart-Multi-Vendor-Marketplace>

<https://github.com/abdulnabi110-blip/Smart-Multi-Vendor-Marketplace>

<https://github.com/hasnain1122730/Smart-Multi-Vendor-Marketplace>

9. Challenges and Lessons Learned

During the development of the Smart Multi-Vendor Marketplace project, several challenges were faced. Initial difficulties included MongoDB Atlas connection issues, authentication errors, and frontend-backend integration problems. Understanding aggregation pipelines, indexing, and query optimization also required careful practice and debugging. Frontend responsiveness and dashboard design improvements were additional challenges during development.

Through this project, important concepts were learned including MERN stack development, MongoDB CRUD operations, aggregation pipelines, indexing, transactions, API development, React state

management, and MongoDB Atlas integration. The project significantly improved practical understanding of NoSQL databases and full-stack web development.

Conclusion and Future Enhancements

The Smart Multi-Vendor Marketplace project was successfully developed using MongoDB, Express.js, React.js, and Node.js technologies. The system performs product management, order handling, analytical reporting, and vendor analytics through a modern dashboard interface.

Advanced MongoDB concepts including aggregation pipelines, indexing, query optimization, views, transactions, and Atlas integration were implemented successfully. The project demonstrated how MongoDB efficiently manages flexible marketplace data using NoSQL architecture.

Future enhancements may include:

- User authentication system
- Admin dashboard
- Payment gateway integration
- Product image upload
- AI-based recommendations
- Advanced analytics
- Mobile application support

Overall, the project achieved its objectives and provided valuable practical experience in MongoDB and MERN stack development.